

LIAR: Liar Interrogates AI Responses

A Wikidata-Based Approach to Claim Verification

Tobias Bonauer

Henrik Obach

Julian Tschurtschenthaler

Abstract

This paper presents LIAR (Liar Interrogates AI Responses), a system for automated verification of natural-language statements using Wikidata. LIAR combines large language models with symbolic reasoning methods to translate unstructured statements into queries that can be evaluated against a structured knowledge graph. Instead of relying on an LLM to determine whether a statement is true or false, the final verification is based on evidence retrieved from Wikidata. The system consists of a verification backend and a web-based frontend for submitting text and presenting the verification results. This paper describes the design and implementation of both components, demonstrates the system through representative examples and discusses its main limitations. The code is available at <https://github.com/Hobac/LIAR>.

1 Introduction

Large language models (LLMs) can generate fluent and convincing natural-language responses, but the factual correctness of their statements cannot be guaranteed. In particular, incorrect information may be presented with the same confidence and linguistic quality as correct information. This creates a need for methods that can automatically detect and verify factual claims in arbitrary natural-language text.

Knowledge graphs provide a promising basis for such verification. Wikidata is such a graph and contains a large collection of facts about entities and their relations and provides access to this information through SPARQL queries. However, directly using a knowledge graph for fact verification introduces a fundamental gap. Claims are expressed in unrestricted natural language, whereas knowledge graphs represent information using predefined graph structures. A verification system therefore has to identify factual claims, determine their intended meaning, translate them into a formal rep-

resentation and connect the concepts of the formal structure to the corresponding knowledge-graph elements.

In this work, we present *LIAR (Liar Interrogates AI Responses)*, a system for automatically verifying natural-language claims against Wikidata. LIAR combines the language-understanding capabilities of LLMs with classic symbolic reasoning methods. LLMs are used to extract and disambiguate factual statements, translate them into a restricted logical representation and assist in resolving natural-language entities and relations to Wikidata identifiers. The resulting expressions are then deterministically compiled into SPARQL queries and evaluated against Wikidata.

A central design goal is to separate the interpretation of natural language from the actual verification step. The LLM determines how a statement should be represented, but it does not decide whether the statement is true or false. Instead, the truth value is derived from the results returned by Wikidata. This allows the system to provide an actual proof alongside the truth value.

The remainder of this paper first reviews related approaches to automated fact verification and presents the architecture of LIAR, including both its frontend and the individual stages of the verification backend. The system’s operation is then illustrated through several examples. Finally, we discuss its limitations, including those arising from Wikidata’s coverage and the chosen logical representation, before outlining possible directions for future work.

2 Background

LIAR represents natural-language claims using a restricted subset of first-order logic (FOL). We assume familiarity with the basic concepts of first-order logic, including predicates, variables, logical connectives such as $\wedge$, $\vee$ and $\neg$, as well as existen-

tial and universal quantification.

A detailed introduction to first-order logic is outside the scope of this report. Readers unfamiliar with these concepts are referred to (Smullyan, 1968) and (Harrison, 2009) for a comprehensive introduction.

LIAR uses Wikidata as its underlying knowledge graph. For an introduction to Wikidata and its data model we refer the reader to the official Wikidata documentation (Wikimedia Foundation).

3 Related Work

Knowledge graphs have previously been explored as structured evidence sources for automated fact verification. FactKG (Kim et al., 2023) introduces a dataset of natural-language claims that can be verified through reasoning over knowledge graphs. The dataset covers several reasoning types, including one-hop reasoning, conjunction, existence, multi-hop reasoning and negation. The work demonstrates the potential of knowledge graphs for fact verification.

Gautam and Pop (Gautam and Pop, 2024) propose FactGenius, which uses LLMs and fuzzy text matching to identify relevant relations in a knowledge graph and determine whether they support a claim.

Hao and Wu (Hao and Wu, 2025) propose Programmatic Graph Reasoning (PGR), which uses LLMs and executable graph programs to verify claims.

LIAR follows the same general direction of combining language models with structured knowledge-graph reasoning, but places a strong separation between natural-language interpretation and verification. First-order logic is used as an intermediate representation because its predicates, logical connectives and quantifiers closely reflect the structure of natural-language claims, making it suitable for LLM-based translation. At the same time, the explicit formula can be presented to the user, making transparent exactly which interpretation of the original claim was verified. This, together with the evidence retrieved from Wikidata, is intended to make the verification process more understandable and trustworthy.

4 System Architecture of LIAR

LIAR consists of a frontend for user interaction and a backend responsible for the verification process. The frontend allows users to submit natural-

language text and presents the verification results, while the backend processes the submitted text, verifies the extracted claims against Wikidata and returns the corresponding truth values and evidence. The following sections describe both components in detail.

4.1 Frontend: Design Principles and User Experience

The frontend provides a focused interface for verifying AI-generated or other natural-language text. A large text field forms the primary interaction area, while a prominent verification button establishes a clear workflow. The dark, blue-toned design reduces visual clutter and uses high-contrast green, yellow and red text to distinguish true, unknown and false claims. Verification statistics summarize the results below the input.

Claims are highlighted directly within the submitted text, preserving their original context. Hovering over a claim displays the proof returned by the backend, including clickable source links to the provided evidence. The interface provides immediate feedback about backend availability, processing state and connection errors. The consistent visual feedback, transparent presentation of supporting evidence and direct links to sources are intended to build user trust and make each verification result understandable and traceable.

4.2 Backend: From Natural-Language Claims to Wikidata Evidence

The backend implements the verification component of LIAR. It accepts English text, extracts factual claims, translates them into a restricted logical representation and searches Wikidata for evidence. For each successfully processed claim, it returns a truth value and proof if the claim could be verified as true or false. The frontend provides access to this functionality through an HTTP API.

The central idea is to separate language interpretation from evidence retrieval. An LLM interprets the wording of a claim and maps it to a structured representation. The truth value is then derived from the result of a knowledge-graph query rather than directly requested from the LLM. Nevertheless, the verification still depends on the correctness of the model’s interpretation and entity selection. We will now explore the individual steps of the verification process.

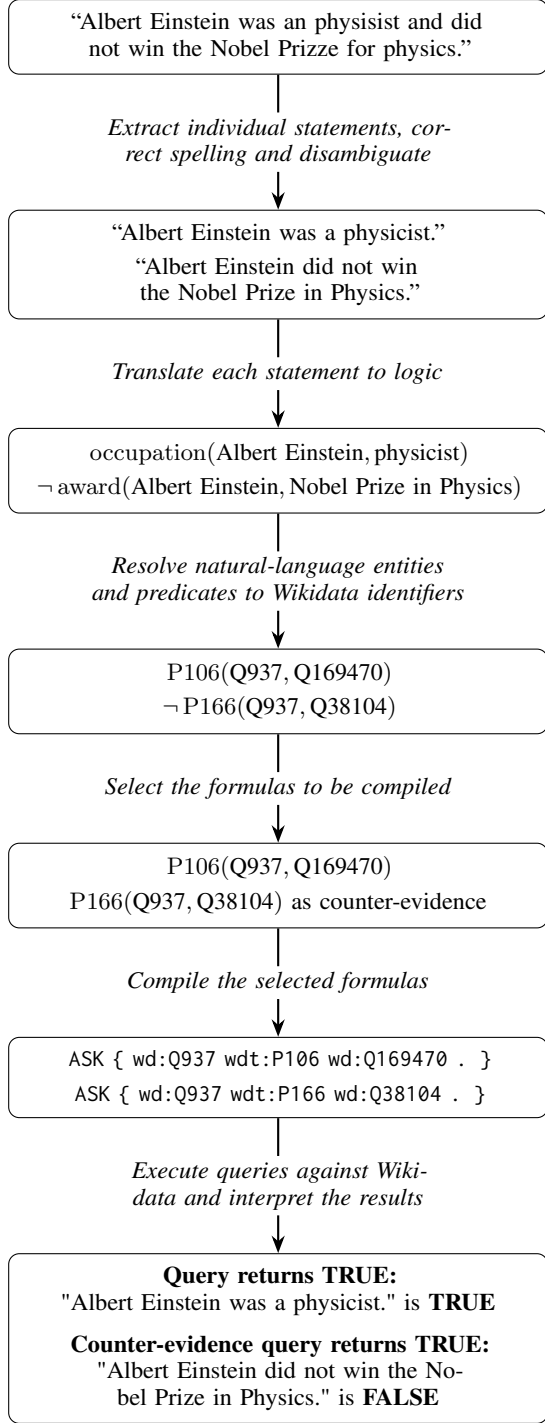


Figure 1: Example execution of the pipeline.

Input Text. The system receives arbitrary English text through the backend API. The input may contain multiple sentences and claims.

Claim Extraction and Disambiguation. An LLM identifies claims in the text that are suitable for verification. The LLM is instructed to not fact-check and just extract all claims that are made. For each claim, the original wording is stored and an unambiguous context-independent version is gen-

erated in which references, such as pronouns, are made explicit. It is then checked if the extracted statements are actually contained in the input text to catch LLM hallucinations.

Logical Representation. Each disambiguated claim is translated by the LLM into a restricted first-order logic formula. LIAR is capable of processing formulas if they either have no variables or if every variable is bound. Free variables are not possible. Additionally quantifiers cannot be mixed, so every variable has to be bound by the same type of quantifier, either universal or existential. For example, $\text{bornIn}(\text{Einstein}, \text{Germany})$ contains no variables and is supported, while $\exists x (\text{occupation}(x, \text{physicist}) \wedge \text{bornIn}(x, \text{Germany}))$ contains a bound variable and is also supported. In contrast, mixed quantifiers such as $\forall x \exists y \text{parentOf}(x, y)$ and free variables such as in $\exists x \text{bornIn}(x, y)$ are not supported.

These restrictions limit the search space of the generated queries and reduce the reliance on closed-world assumptions. Since LIAR is given a fixed interpretation and domain through Wikidata, possible bindings have to be found by searching the knowledge graph. Free variables may therefore result in insufficiently constrained queries that require searching over large parts of the graph.

Mixed quantifiers, such as $\forall x \exists y P(x, y)$, can similarly require very large searches to find suitable bindings for each entity. Furthermore, verifying such a formula requires finding a suitable binding for y for every possible binding of x . If no suitable y can be found for just one x , the entire universal statement would have to be rejected. Under Wikidata’s open-world assumption, however, the absence of such a binding does not establish that none exists. The required information may simply be missing from Wikidata. Treating one missing binding as sufficient to reject the statement would therefore require a closed-world assumption and therefore could lead to misleading or incorrect results.

The LLM is instructed to reject the statement if it cannot be expressed in the given logic. The LLM outputs the formula in JSON. A parser is used to check the output syntax. Additionally the intended logical structure of the JSON output is checked by a checker class that provides detailed feedback to the LLM if the structure was violated.

Wikidata Resolution. At this step the formula still contains predicates and constants in natural language. This language must be mapped to Wikidata identifiers in order to query the database. Wikidata provides an API that resolves any natural language term into a list of suggested identifiers. Using this API, it is possible to resolve both entities like *Albert Einstein* that are represented as vertices in the knowledge graph and relations like *award* which form the edges. Candidate identifiers are listed in a prompt to an LLM that is tasked with selecting those that best represent the meaning of the natural language in the formula. For a broader coverage, synonyms for each relevant term are also resolved. These synonyms are retrieved from an LLM that is provided the term and the context.

Since not every relation between two entities is represented as a direct edge in Wikidata we have introduced more complex relations with a hand-crafted list. For example, the relation $\text{relative}(x, y)$ is not included directly in Wikidata. There are edges for $\text{father}(x, y)$, $\text{child}(x, y)$, $\text{sibling}(x, y)$ though. Thus one can construct the relation $\text{relative}(x, y)$ as the transitive closure of these relations, meaning two entities are relatives if they can reach each other in the graph using the provided edge types. Such complex predicates are crafted manually and are stored in a list with synonyms that are used for the resolution. So $\text{relative}(x, y)$ could be in relation to words like *relative*, *related to*, *family member of* etc.

Formula Selection. For simplicity, expressions such as $P(x)$ are used schematically to denote predicates involving x , the actual predicates handled by LIAR are binary.

After resolution, LIAR determines which formula should be passed to the query compiler. The goal is to generate a query for which finding a model provides evidence either for or against the original claim. If the resolved formula contains no negation, it can be compiled directly, since every condition corresponds to information that must explicitly exist in Wikidata. A returned model therefore provides positive evidence for the claim, while the absence of a model only results in Unknown under the open-world assumption.

For formulas containing negation, LIAR attempts to pull an existing negation to the outside of the expression so that the remaining inner formula is positive and can be queried for counter-evidence. The transformation uses De Morgan’s

laws, in particular $\neg A \wedge \neg B \equiv \neg(A \vee B)$ and $\neg A \vee \neg B \equiv \neg(A \wedge B)$. Thus, $\neg P(x) \wedge \neg Q(x)$ may be rewritten as $\neg(P(x) \vee Q(x))$, allowing LIAR to query the positive formula $P(x) \vee Q(x)$. If a model is found, it disproves the original claim. If no model is found, the result remains Unknown.

Crucially, LIAR only pulls negations outward that already occur in the formula. It must not introduce a new double negation merely to create such a query. For example, the mixed formula $P(x) \wedge \neg Q(x)$ must not be wrapped as $\neg\neg(P(x) \wedge \neg Q(x))$ in order to force an outer negation. Pushing the inner negation inward would yield $\neg(\neg P(x) \vee Q(x))$, whose inner formula still contains the negative literal $\neg P(x)$. Evaluating such a negative condition could succeed merely because the corresponding fact is absent from Wikidata. Under the open-world assumption, however, a missing fact represents unknown information rather than falsity.

Universal statements form an exception to the open-world assumption because they cannot be established directly from an incomplete knowledge graph. Instead, LIAR searches for a concrete counterexample. For example, $\forall x (P(x) \rightarrow Q(x))$ is transformed into the counterexample formula $\exists x (P(x) \wedge \neg Q(x))$ and compiled directly. In this case, the negative condition $\neg Q(x)$ is evaluated under a closed-world assumption. Meaning the absence of the corresponding $Q(x)$ relation in Wikidata is treated as sufficient for the counterexample search.

This exception is necessary because otherwise universal statements containing such negative counterexample literals could never be disproven using Wikidata. The assumption is used only to search for a concrete counterexample and is explicitly recorded in the generated proof. Failure to find such a counterexample does not establish the universal statement, it still results in Unknown.

SPARQL Query Compilation. The resolved formula is transformed into SPARQL by the query compiler. If the formula is universally quantified, it is transformed using $\forall x \varphi \equiv \neg \exists x \neg \varphi$, thereby turning it into an existential search for a counterexample. Now we can treat all variables occurring in the formula as existentially quantified.

The expression is now simplified to a formula using only $\wedge$, $\vee$ and $\neg$ as logical operators. It is then converted to negation normal form (NNF) and subsequently to disjunctive normal form (DNF).

The DNF representation exposes the individual conjunctions of the formula, allowing them to be optimized by removing duplicate literals, unsatisfiable conjunctions containing both a literal and its negation and duplicate conjunctions before the final SPARQL query is constructed.

DNF is particularly useful because negative literals require bindings that can preferably be supplied by positive literals in the same conjunction. For example, the SPARQL pattern corresponding to $P(x) \wedge \neg Q(x)$ first uses $P(x)$ to find matching entities and bind x . The condition $\neg Q(x)$ can then be checked for each of these bindings using `FILTER NOT EXISTS`. Without the positive literal, $\neg Q(x)$ provides no binding for x itself, and the query would have no set of entities over which to evaluate the negative condition.

For example, $P(x) \wedge (\neg Q(x) \vee \neg R(x))$ is transformed into $(P(x) \wedge \neg Q(x)) \vee (P(x) \wedge \neg R(x))$. Each resulting conjunction contains the positive literal $P(x)$, which can provide a binding for x before the negative literals are evaluated.

For conjunctions consisting exclusively of negative literals, the compiler introduces a fallback binding for each occurring variable using Wikidata's instance of property P31, which is a very general property that most entities have. This is necessary because negative graph patterns cannot bind variables themselves. A positive SPARQL triple pattern such as `?x wdt:P31 ?type`, is needed to bind the variable `?x`. A negative pattern using `FILTER NOT EXISTS`, can then check whether a matching triple exists for an already bound variable.

Adding the fallback restricts the search to entities having a P31 statement, which may cause an existing model to be missed, but still allows models within this restricted domain to be found. Since the compiler only searches for a model and does not interpret the absence of one as proof of unsatisfiability, this restriction is acceptable.

The fundamental compiler rules are

$$\begin{aligned} P(x, y) &\longmapsto x \text{ wdt:P } y \text{ .,} \\ x \theta y &\longmapsto \text{FILTER}(x \theta y), \\ \varphi \wedge \psi &\longmapsto \{ \varphi \psi \}, \\ \varphi \vee \psi &\longmapsto \{ \varphi \} \text{ UNION } \{ \psi \}, \\ \neg \varphi &\longmapsto \text{FILTER NOT EXISTS } \{ \varphi \}, \end{aligned}$$

A predicate $P(x, y)$ is translated into a SPARQL triple, where P is the resolved Wikidata property and x, y represent variables or entities. For example, $P106(Q937, Q169470)$ be-

comes `wd:Q937 wdt:P106 wd:Q169470 .`, while $P106(x, Q169470)$ becomes `?x wdt:P106 wd:Q169470 .`

A comparison $x \theta y$ is translated into a SPARQL `FILTER`, where $\theta \in \{=, ! =, <, <=, >, >=\}$ denotes a SPARQL comparison operator. For example, $a > b$ becomes `FILTER(?a > ?b)`, while $a \leq 50$ becomes `FILTER(?a <= 50)`. The variables used in a comparison have to be bound by predicates elsewhere in the query.

A conjunction $\varphi \wedge \psi$ is translated by concatenating both expressions in the query, requiring both expressions to hold. A disjunction $\varphi \vee \psi$ is translated using `UNION`, allowing either expression to hold. Finally, a negation $\neg \varphi$ is translated using `FILTER NOT EXISTS`, which succeeds if no matching result for φ exists.

Finally, the type of SPARQL query depends on whether variables occur in the generated expression. If variables remain, the compiler generates `SELECT * WHERE { $\varphi$ } LIMIT 1` since one matching element is sufficient. If the expression contains no variables, the compiler instead generates `ASK { $\varphi$ }` since only the existence of a matching graph pattern has to be determined.

Wikidata Query. The generated SPARQL query is executed against Wikidata. If a return value exists, it is stored as evidence and used to construct the proof.

Truth Value and Evidence. Finally, the query result is interpreted as True, False, or Unknown. For positive claims, the system follows the open-world assumption: a claim is True if supporting evidence is found and Unknown otherwise. For example, if Wikidata confirms that Albert Einstein was a physicist, the claim “Albert Einstein was a physicist” is True. The claim “Albert Einstein was not a physicist” is False, since evidence for its positive counterpart exists. If no evidence exists for either side, the result remains Unknown.

As mentioned, universal claims are handled differently by searching for counterexamples using $\neg \forall x \varphi(x) \equiv \exists x \neg \varphi(x)$. If a counterexample is found, the claim is False. Otherwise it remains Unknown. This search relies on a closed-world assumption, since a missing relation in Wikidata may be treated as evidence for a counterexample.

When available, evidence is returned alongside the formula and its truth value.

5 Results

We will now examine some illustrative examples that show the capabilities of LIAR. Each example shows the natural-language claim, its logical representation and how the resulting formula is interpreted during verification. We omit the compiled query here, since the compilation is deterministic.

Positive claim. Consider the statement “Albert Einstein was a physicist” represented as

$$\text{occupation}(\text{Einstein}, \text{physicist}).$$

The formula is queried directly against Wikidata. If the corresponding relation is found, the claim is True. Otherwise, it remains Unknown.

Negative claim. Consider the statement “Albert Einstein did not win the Nobel Prize in Physics” represented as

$$\neg \text{award}(\text{Einstein}, \text{Nobel Prize in Physics}).$$

LIAR queries the positive inner formula. If the corresponding award relation is found, the claim is False. Otherwise, it remains Unknown.

Conjunction. Consider the statement “Albert Einstein was a physicist and was born in Germany” represented as

$$\begin{aligned} &\text{occupation}(\text{Einstein}, \text{physicist}) \\ &\wedge \text{bornIn}(\text{Einstein}, \text{Germany}). \end{aligned}$$

Both predicates are queried together against Wikidata. If the corresponding relations are found, the complete claim is True. Otherwise, it remains Unknown. Other logical operators are omitted here, as they are handled analogously, as described by the compiler rules in 4.2. They behave as expected from their standard logical semantics.

Comparison. Consider the statement “Germany has a larger population than Austria” represented as

$$\begin{aligned} &\text{population}(\text{Germany}, ?a) \\ &\wedge \text{population}(\text{Austria}, ?b) \\ &\wedge ?a > ?b. \end{aligned}$$

For comparisons, the LLM introduces variables for the values being compared. The variables are bound by corresponding predicates. In this example, ?a is bound to the population of Germany and ?b to the population of Austria. If the variables can be bound and the comparison is true, the claim is True. Otherwise, it remains Unknown.

Existential claim. Consider the statement “There exists a physicist who won the Nobel Prize in Physics” represented as

$$\exists x (\text{occupation}(x, \text{physicist}) \wedge \text{award}(x, \text{Nobel Prize in Physics})).$$

The query searches for an entity that satisfies both predicates. If at least one such entity is found, the claim is True. Otherwise, it remains Unknown. Since the statement is true, a physicist such as Albert Einstein would be provided as positive evidence in the proof.

Universal claim. Consider the statement “All physicists were born in Germany” represented as

$$\forall x (\text{occupation}(x, \text{physicist}) \rightarrow \text{bornIn}(x, \text{Germany})).$$

Instead of attempting to prove the universal statement directly, LIAR searches for a counterexample by transforming the formula into

$$\begin{aligned} &\exists x (\text{occupation}(x, \text{physicist}) \wedge \\ &\neg \text{bornIn}(x, \text{Germany})). \end{aligned}$$

If such a counterexample is found, the original universal claim is False. For example, the query could return the Wikidata link to Robert Oppenheimer as a counterexample, which is then provided in the proof as evidence that the universal statement is false. If no counterexample is found, the claim remains Unknown.

The negative predicate in the counterexample is evaluated under a closed-world assumption. Thus, for an entity known to be a physicist, the absence of a corresponding $\text{bornIn}(x, \text{Germany})$ relation in Wikidata is treated as satisfying the negative predicate. This allows LIAR to find counterexamples despite Wikidata’s incomplete information. However, a missing relation may also result from incomplete data and therefore does not necessarily represent an actual negative fact. For this reason, a remark indicating the use of the closed-world assumption is added to the generated proof whenever such a counterexample is returned.

6 Discussion

LIAR demonstrates that natural-language claims can be translated into a formal representation and verified against a knowledge graph without asking an LLM to determine their truth directly. However,

the reliability and coverage of the system are limited by both the representation of information in Wikidata and the translation from natural language to first-order logic.

One limitation is that Wikidata does not always represent information in the form expected by a direct translation of a claim. For example, the statement “Albert Einstein died in 1955” cannot simply be translated into a relation between Albert Einstein and the value 1955. Wikidata stores the date of death as a complete date with a specific data type. Correctly handling such claims therefore requires additional rules for translating and comparing Wikidata values. Similar problems can occur with other data types and comparisons. Consequently, a query may fail not because the claim is false, but because the generated query does not correctly reflect how the information is represented in Wikidata. To account for this, LIAR never interprets a failed query involving comparisons as evidence that a claim is false. Such cases instead result in Unknown, preserving the soundness of the verification process.

The use of a restricted first-order logic representation introduces another limitation. LIAR supports formulas without variables and formulas in which all variables are bound and use the same type of quantifier. Mixed quantification, such as $\forall x \exists y \text{ parentOf}(x, y)$, cannot be processed. This restriction reduces the range of natural-language statements that can be verified.

Wikidata’s incomplete nature also limits the conclusions that can be drawn from query results. Under the open-world assumption, failure to find evidence for a claim does not imply that the claim is false. LIAR therefore returns Unknown whenever neither supporting nor contradicting evidence can be established. While this avoids treating missing information as false information, it also means that many claims cannot receive a definite truth value.

Universal claims require an additional compromise. LIAR searches for counterexamples to such claims and evaluates negative conditions in these searches under a closed-world assumption. This makes it possible to find potential counterexamples, but a missing relation in Wikidata may result from incomplete data rather than an actual negative fact. Counterexamples obtained in this way must therefore be interpreted with care, which is why LIAR explicitly indicates the use of this assumption in the generated proof.

Although the final verification is based on Wiki-

data rather than an LLM judgment, the system is not independent of LLM errors. The LLM is responsible for extracting claims, translating them into logical formulas and assisting with the resolution of entities and predicates. An incorrect interpretation at any of these stages can therefore lead to a query that correctly evaluates the wrong formula. Presenting the generated formula and retrieved evidence to the user helps make such errors visible instead of hiding the interpretation behind a final truth value.

Finally, the correspondence between natural-language relations and Wikidata properties is not always direct. Some concepts require combinations of multiple Wikidata properties. For example, the concept of a *relative* can be represented by combining properties such as *parent*, *child* and *sibling*. LIAR supports such cases through manually defined complex predicates, but their coverage is necessarily limited.

Overall, LIAR trades coverage for a verification process that is explicit and traceable. Treating missing evidence conservatively prevents many claims from receiving a definite result, but allows successful verification results to be reliable.

7 Conclusions and Future Work

This work presented LIAR, a fact-verification system that combines large language models, first-order logic and Wikidata. Rather than asking a language model to directly judge the truth of a statement, LIAR uses the model to translate natural-language claims into a structured logical representation. The resulting formulas are translated into SPARQL queries whose results determine the final truth value.

The current implementation demonstrates that this approach can represent and verify a wide range of claims. At the same time, the limitations discussed in the previous section leave several opportunities for improving the system.

Extending the translation pipeline to support mixed quantifiers would increase the range of natural-language statements that can be expressed. This would require suitable approximations to make their evaluation feasible.

Comparisons could also be improved by distinguishing between a comparison that evaluates to false and one that fails because the values have incompatible data types. This distinction could then be considered when determining the final truth

value.

Finally, the logical reasoning stage could be strengthened by integrating dedicated logic solvers. Although first-order logic is undecidable in general, formulas belonging to decidable fragments could be identified and handled by a SAT or SMT solver. This could detect unsatisfiable formulas before SPARQL queries are even generated.

Overall, LIAR demonstrates how language models and structured knowledge bases can work together in automated fact verification. Language models provide the flexibility required to interpret natural-language statements, while the logical representation and Wikidata provide sound and inspectable reasoning.

References

- Sushant Gautam and Roxana Pop. 2024. FactGenius: Combining zero-shot prompting and fuzzy relation mining to improve fact verification with knowledge graphs. In *Proceedings of the Seventh Fact Extraction and VERification Workshop (FEVER)*, pages 297–306, Miami, Florida, USA. Association for Computational Linguistics.
- Yuanzhen Hao and Desheng Wu. 2025. Fact verification on knowledge graph via programmatic graph reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5480–5495, Suzhou, China. Association for Computational Linguistics.
- John Harrison. 2009. *Handbook of Practical Logic and Automated Reasoning*. Cambridge University Press.
- Jiho Kim, Sungjin Park, Yeonsu Kwon, Yohan Jo, James Thorne, and Edward Choi. 2023. FactKG: Fact verification via reasoning on knowledge graphs. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16190–16206, Toronto, Canada. Association for Computational Linguistics.
- Raymond Merrill Smullyan. 1968. *First-Order Logic*. Springer Verlag, New York [etc.].
- Wikimedia Foundation. Wikidata. https://www.wikidata.org/wiki/Wikidata:Main_Page. Accessed: 2026-09-28.